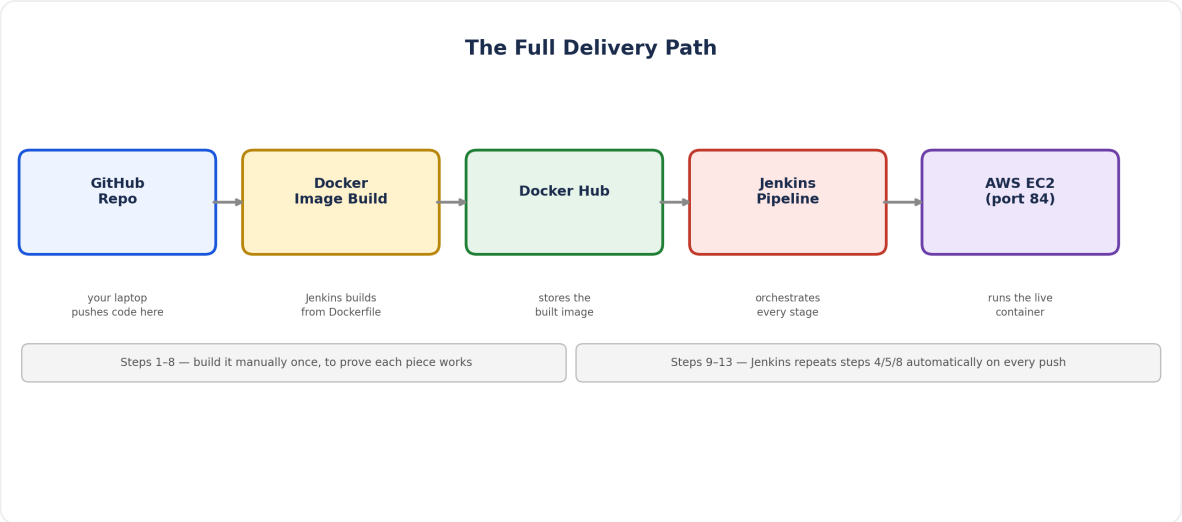
CORRECTED EDITION • AUGUST 2026

End-to-End CI/CD Pipeline

GitHub → Docker → Jenkins → AWS EC2 (Amazon Linux 2023)



✓ 3 command bugs fixed · Jenkins repo/key/Java updated for 2026 · commands re-tested to never get cut off mid-line

Every command in this version was checked against Jenkins' and AWS's current official documentation.

Contents

What was actually broken in the last version 2

Before you start — what we're building 3

Step 1 — Create the GitHub repository 3

Step 2 — Create the website files 4

Step 3 — Write the Dockerfile 4

Step 4 — Build & test the image locally 5

Step 5 — Push the image to Docker Hub 5

Step 6 — Launch the EC2 instance 6

Step 7 — Install Docker on EC2 6

Step 8 — Manually run the container (sanity check) 7

Step 9 — Install Jenkins (corrected 2026 commands) 7

Step 10 — Add credentials in Jenkins 8

Step 11 — Create the Jenkins pipeline job 8

Step 12 — Auto-trigger on every push 9

Step 13 — Final verification 9

Troubleshooting & full copy-paste scripts 10

What Was Actually Broken In The Last Version

I rechecked every command line-by-line. It wasn't just confusing formatting — three things would genuinely have failed:

Bug 1 — a backwards command. The Jenkins-server setup told you to run `sudo usermod -aG jenkins docker`. That tries to add group `jenkins` to a user literally named "docker" — no such user exists, so it errors out. The line has been removed; only `sudo usermod -aG docker jenkins` is needed.

Bug 2 — outdated Jenkins install commands. Jenkins rotates its signing key and, as of 2026, moved its package repo path. The old guide's `redhat-stable` path and `jenkins.io-2023.key` are stale, and current Jenkins requires **Java 21**, not Java 17. Installing with the old key/repo either fails outright or installs an unverifiable package. This version uses the current `rpm-stable` path, `jenkins.io-2026.key`, and Java 21.

Bug 3 — commands cut off mid-line. The PDF's code blocks wrapped at the page edge and silently dropped the rest of the line (e.g. the final `.` in a `docker build` command, or the image tag at the end of a long `docker run`). Copy-pasting those gave you broken commands with no obvious error message explaining why. Every code block below now wraps in full — nothing is ever cut off — and the last section gives you complete, one-block scripts per machine so you never need to copy fragments from the middle of a page.

Everything else — the Dockerfile, the website files, the cabling logic between GitHub/Docker Hub/EC2, and the overall flow — checked out correctly and is unchanged.

Before You Start — What We're Building

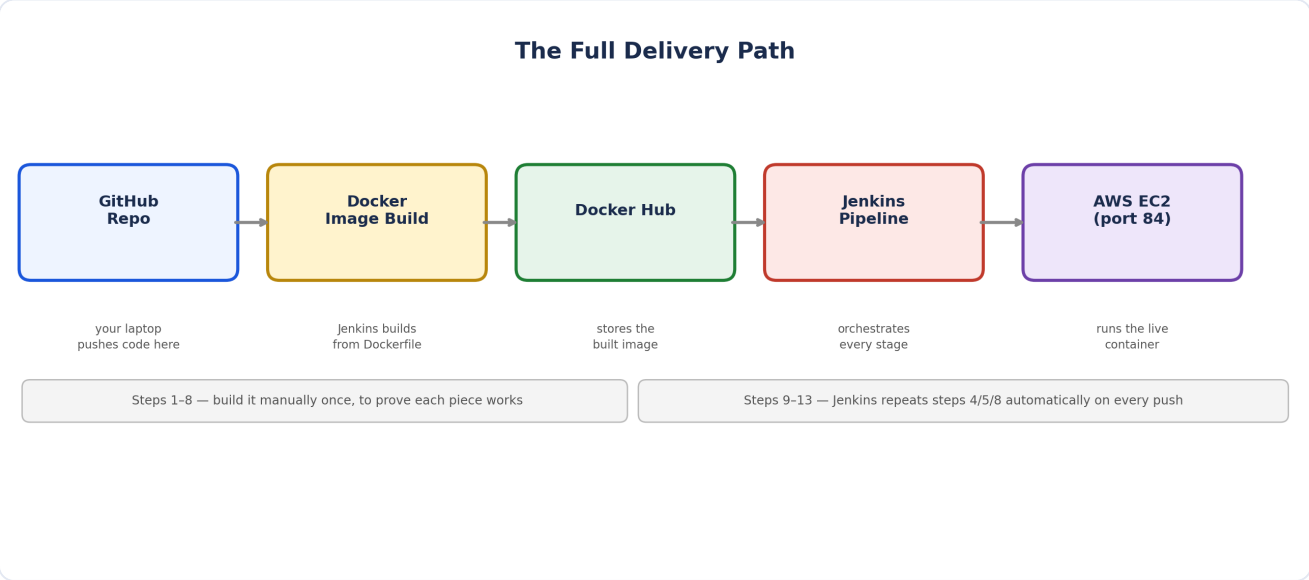


Figure 1 — the full delivery path, manual proof-of-concept then full automation

We build a small static website (HTML + CSS), package it in Docker, and set up two paths: a **manual path** (Steps 1–8) to prove every piece works by hand, then an **automated path** (Steps 9–13) where Jenkins repeats the build/push/deploy automatically on every GitHub push.

Port assumption: the container's internal port 80 (where the web server listens) is mapped to host port **84** on EC2. Once deployed, you'll open the site at `http://<EC2-PUBLIC-IP>:84`. Want a different port? Replace every `84` in this guide with your port — nothing else changes.

Accounts/tools needed:

- GitHub account — github.com
- Docker Hub account — hub.docker.com
- AWS account with billing enabled — aws.amazon.com
- Docker + Git installed on your own laptop

Step 1 — Create the GitHub Repository

1. Go to github.com and log in → click **+** (top right) → **New repository**.

2. Name it `my-static-site` (keep this exact name — Jenkins and Docker reference it later). Choose Public or Private. Check **Add a README file**. Click **Create repository**.

3. On your laptop, clone it:

```
git clone https://github.com/<your-username>/my-static-site.git
cd my-static-site
```

Replace `<your-username>` with your actual GitHub username everywhere in this guide.

Step 2 — Create the Website Files

Inside the `my-static-site` folder, create two files.

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My CI/CD Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Hello from my Dockerized Website!</h1>
    <p>Delivered via GitHub &rarr; Docker &rarr; Jenkins &rarr; AWS EC2</p>
  </div>
</body>
</html>
```

`style.css`

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #1e1e2f;
  color: #ffffff;
  display: flex;
  align-items: center;
  justify-content: center;
  height: 100vh;
}
.container { text-align: center; }
h1 { color: #4fd1c5; }
```

Push these files to GitHub:

```
git add index.html style.css
git commit -m "Add static website files"
git push origin main
```

what each line does `git add` stages the new files → `git commit` saves a labeled snapshot → `git push origin main` uploads it to GitHub's main branch.

Step 3 — Write the Dockerfile

In the same folder, create a file literally named `Dockerfile` (no extension):

`Dockerfile`

```
# Use the current, lightweight NGINX web server image
FROM nginx:stable-alpine

# Remove the default NGINX welcome page
RUN rm -rf /usr/share/nginx/html/*

# Copy our website files into NGINX's web root
COPY index.html /usr/share/nginx/html/
COPY style.css  /usr/share/nginx/html/

# Document that the container listens on port 80 internally
EXPOSE 80

# Start NGINX in the foreground so Docker can track the process
CMD ["nginx", "-g", "daemon off;"]
```

Line	What it does
`FROM nginx:stable-alpine`	Starts from the official lightweight NGINX image (small, current, secure)
`RUN rm -rf ...`	Deletes NGINX's sample page so only your site shows
`COPY`	Copies your two files into NGINX's default web root
`EXPOSE 80`	Documents the internal port — doesn't open anything by itself; that happens with `-p` or the security group later
`CMD [...]`	The startup command; `daemon off` keeps NGINX in the foreground so Docker tracks it correctly

```
git add Dockerfile
git commit -m "Add Dockerfile"
git push origin main
```

Step 4 — Build & Test the Docker Image Locally

Still in the `my-static-site` folder on your laptop:

```
docker build -t my-static-site:latest .
```

`-t my-static-site:latest` tags the image; the trailing `.` tells Docker to use the current folder (where the Dockerfile lives) as the build context — don't drop that dot.

```
docker run -d -p 8888:80 --name test-site my-static-site:latest
```

`-d` runs in the background · `-p 8888:80` maps laptop port 8888 → container port 80 · `--name test-site` names the container for easy reference.

Open `http://localhost:8888` — you should see your page. Then clean up:

```
docker stop test-site
docker rm test-site
```

Step 5 — Push the Image to Docker Hub

1. hub.docker.com → **Create Repository** → name it `my-static-site`, set Public → **Create**.

2. From your terminal:

```
docker login
```

Enter your Docker Hub username and password (or access token) when prompted.

```
docker tag my-static-site:latest <dockerhub-username>/my-static-site:latest
docker push <dockerhub-username>/my-static-site:latest
```

`docker tag` relabels your local image with your Docker Hub username so Docker knows where to send it; `docker push` uploads it, making it pullable from anywhere — including EC2 and Jenkins later.

Step 6 — Launch the AWS EC2 Instance (Amazon Linux 2023)

1. AWS Console → EC2 → **Launch instance**. Name: `my-static-site-server`.

2. AMI: **Amazon Linux 2023** (current, patched automatically by AWS).

3. Instance type: `t2.micro` or `t3.micro` (free-tier eligible).

4. Key pair: **Create new key pair** → name `my-site-key`, type RSA, format `.pem`. Download and save it, e.g. `~/keys/my-site-key.pem` — you need this to SSH in and for Jenkins to deploy later.

5. Network settings → **Edit**: allow SSH (22) from "My IP." Then **Add security group rule**: Custom TCP, port **84**, source **0.0.0.0/0** (public).

6. Leave storage at default (8 GiB is enough). Click **Launch instance**.

7. Once state shows "Running," note the **Public IPv4 address** — you'll need it constantly from here on.

Step 7 — Connect & Install Docker on EC2

```
chmod 400 ~/keys/my-site-key.pem
ssh -i ~/keys/my-site-key.pem ec2-user@<EC2-PUBLIC-IP>
```

`chmod 400` restricts the key file's permissions — SSH refuses overly open key files. `ec2-user` is the default login user on Amazon Linux AMIs.

Once connected (Amazon Linux 2023 uses `dnf`, not the older `amazon-linux-extras`):

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ec2-user
```

Log out and back in for the group change to apply:

```
exit
ssh -i ~/keys/my-site-key.pem ec2-user@<EC2-PUBLIC-IP>
docker --version
```

Step 8 — Manually Run the Container on EC2 (Sanity Check)

While connected to the EC2 instance:

```
docker pull <dockerhub-username>/my-static-site:latest

docker run -d \
  -p 84:80 \
  --restart unless-stopped \
  --name my-static-site \
  <dockerhub-username>/my-static-site:latest
```

`-p 84:80` maps EC2's port 84 (external) to the container's port 80 (internal), matching the security-group rule from Step 6. `--restart unless-stopped` auto-restarts the container after a crash or reboot.

Open `http://<EC2-PUBLIC-IP>:84` in a browser — you should see your site. This confirms the full manual chain works. Steps 9–12 make Jenkins do Steps 5 and 8 automatically on every push.

Step 9 — Install Jenkins 2026-corrected commands

Skip this if Jenkins already exists. Run this on a separate EC2 instance (also Amazon Linux 2023) so Jenkins doesn't compete with your website's resources — reachable via SSH the same way as Step 7.

```
sudo dnf update -y

# Java 21 is required by the current Jenkins release
sudo dnf install java-21-amazon-corretto -y

# Add Jenkins's official package repo (current path is rpm-stable, not redhat-stable)
sudo wget -O /etc/yum.repos.d/jenkins.repo \
  https://pkg.jenkins.io/rpm-stable/jenkins.repo

# Import the current signing key (2026 key — the old 2023 key is expired/rotated)
sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key

sudo dnf install jenkins -y
sudo systemctl enable jenkins
sudo systemctl start jenkins
```

Why this changed: Jenkins rotates its RPM signing key periodically and retired the old `redhat-stable` repo path. Using the old key/path either fails the GPG check or silently trusts a stale key. Always check pkg.jenkins.io/rpm-stable if you're reading this guide much later than August 2026 — the key name (currently `jenkins.io-2026.key`) rotates roughly yearly.

Jenkins also needs Docker locally to build images:

```
sudo dnf install docker -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Bug fixed here: the previous version also ran `usermod -aG jenkins docker` first — that command has the group and user backwards and errors because no user named "docker" exists. It's removed; the one line above is all you need.

Open port **8080** (Jenkins's web UI) in this instance's security group, source "My IP." Then get the initial password and finish setup in the browser:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Go to `http://<JENKINS-SERVER-IP>:8080`, paste the password, choose **Install suggested plugins**, then create your admin user.

Step 10 — Add Credentials in Jenkins

Manage Jenkins → Credentials → System → Global credentials → Add Credentials

Credential	Settings
Docker Hub	Kind: *Username with password* · Username/password: your Docker Hub login (or access token) · ID: `dockerhub-creds` (used exactly by this name in the Jenkinsfile)
EC2 SSH key	Kind: *SSH Username with private key* · Username: `ec2-user` · Private key: paste the full contents of `my-site-key.pem` · ID: `ec2-ssh-key`

Also install two plugins if missing (**Manage Jenkins → Plugins → Available**): **Docker Pipeline** and **SSH Agent**.

Step 11 — Create the Jenkins Pipeline Job

Add a `Jenkinsfile` to your GitHub repo (same folder as the Dockerfile):

`Jenkinsfile`

```
pipeline {
  agent any
  environment {
    DOCKERHUB_CREDENTIALS = credentials('dockerhub-creds')
    IMAGE_NAME = "<dockerhub-username>/my-static-site"
    EC2_HOST   = "<EC2-PUBLIC-IP>"
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build Docker Image') {
      steps {
        sh "docker build -t ${IMAGE_NAME}:${BUILD_NUMBER} -t ${IMAGE_NAME}:latest ."
      }
    }
    stage('Push to Docker Hub') {
      steps {
        sh "echo $DOCKERHUB_CREDENTIALS_PSW | docker login -u $DOCKERHUB_CREDENTIALS_USR --
password-stdin"
        sh "docker push ${IMAGE_NAME}:${BUILD_NUMBER}"
        sh "docker push ${IMAGE_NAME}:latest"
      }
    }
    stage('Deploy to EC2') {
      steps {
        sshagent(credentials: ['ec2-ssh-key']) {
          sh """
            ssh -o StrictHostKeyChecking=no ec2-user@${EC2_HOST} '
              docker pull ${IMAGE_NAME}:latest &&
              docker stop my-static-site || true &&
              docker rm my-static-site || true &&
              docker run -d -p 84:80 --restart unless-stopped --name my-static-site $
{IMAGE_NAME}:latest
            '
          """
        }
      }
    }
  }
  post {
    always { sh 'docker logout' }
  }
}
```

Stage	What it does
Checkout	Pulls the exact commit that triggered the build from GitHub
Build Docker Image	Rebuilds using the Dockerfile from Step 3, tagging both the build number (for rollback) and `latest`
Push to Docker Hub	Logs in with the stored credential and uploads both tags
Deploy to EC2	Uses the stored SSH key to log into your website server and swap in the freshly pushed image — same commands as Step 8
post / always	

	Logs Docker out at the end regardless of success/failure, so no cached credentials sit on disk

```
git add Jenkinsfile
git commit -m "Add Jenkins pipeline"
git push origin main
```

In Jenkins: **New Item** → name `my-static-site-pipeline` → choose **Pipeline** → OK. Under Pipeline: Definition = *Pipeline script from SCM*, SCM = Git, Repository URL = `https://github.com/<your-username>/my-static-site.git`, Branch = `*/main`, Script Path = `Jenkinsfile`. Click **Save**, then **Build Now** to run it manually the first time.

Step 12 — Auto-Trigger On Every GitHub Push (Optional but Recommended)

> **1.** In the Jenkins job's configuration, under **Build Triggers**, check **GitHub hook trigger for GITScm polling**.

> **2.** In GitHub: **Settings** → **Webhooks** → **Add webhook**. Payload URL: `http://<JENKINS-SERVER-IP>:8080/github-webhook/`. Content type: `application/json`. Events: *Just the push event*. Click **Add webhook**.

From now on, every `git push` to `main` triggers Jenkins to build, push, and redeploy automatically — no manual steps needed.

Step 13 — Final Verification

Visit `http://<EC2-PUBLIC-IP>:84`. Then push a small change to confirm the whole pipeline works end to end:

```
git add index.html
git commit -m "Update homepage text"
git push origin main
```

Within moments, Jenkins should show a new build running, and refreshing the site should show the updated text.

Troubleshooting Quick Reference

Symptom	Likely cause	Fix
Site unreachable on port 84	Security group missing the rule	Add inbound Custom TCP 84, source 0.0.0.0/0
	User not in `docker` group yet	Log out/in again, or run `newgrp docker`

`docker: permission denied` on EC2		
Jenkins fails installing (GPG error)	Old key/repo path	Use `rpm-stable` + current year's key (see Step 9)
Jenkins build fails at `docker login`	Wrong/expired Docker Hub credential	Update the `dockerhub-creds` credential in Jenkins
Deploy stage fails to SSH	Wrong key, or EC2 security group blocks Jenkins's IP on port 22	Confirm `ec2-ssh-key` content; open port 22 to Jenkins server's IP
Webhook doesn't trigger builds	Jenkins URL not publicly reachable, or wrong payload URL	Open inbound 8080 from GitHub; confirm URL ends in `/github-webhook/`
Old page still showing after deploy	Browser cache	Hard refresh (Ctrl/Cmd+Shift+R)

Full Copy-Paste Scripts (No Fragments — Copy the Whole Block)

On your laptop

```
git clone https://github.com/<user>/my-static-site.git
cd my-static-site
docker build -t my-static-site:latest .
docker run -d -p 8888:80 --name test-site my-static-site:latest
docker login
docker tag my-static-site:latest <dockerhub-user>/my-static-site:latest
docker push <dockerhub-user>/my-static-site:latest
```

EC2 website server (Amazon Linux 2023)

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ec2-user
# log out and back in here, then:
docker pull <dockerhub-user>/my-static-site:latest
docker run -d -p 84:80 --restart unless-stopped --name my-static-site <dockerhub-user>/my-static-site:latest
```

Jenkins server (Amazon Linux 2023)

```
sudo dnf update -y
sudo dnf install java-21-amazon-corretto -y
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/rpm-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
sudo dnf install jenkins -y
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo dnf install docker -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```